



Challenge : Code Review Agent



Description

Build an **AI agent that reviews code** like a thoughtful senior developer would. Instead of asking an LLM "find bugs in this code" and pasting back whatever it says, your job is to build something more interesting: an agent that combines real analysis tools with an LLM's reasoning to produce a structured, useful code review.

Concretely, your agent should take in a piece of code (a file, a snippet, or even a GitHub PR) and produce a report that flags issues, bugs, security risks, style problems, bad practices, and explains *why* they matter, ideally with suggested fixes.

This is your chance to build something genuinely "agentic": a system that uses tools, reasons over their output, and produces a structured result. That's a skill that shows up everywhere in AI right now, and it's a great thing to point to in an interview.



Steps to Get There

You're free to approach this however makes sense to you, but here's a path that tends to work well:

1. **Pick your scope** : Choose one programming language to support well (Python is the easiest to start with given available tooling). Decide whether you'll review full files or just diffs/PRs, full files are simpler to start with.
2. **Get a baseline signal** : Hook up at least one static analysis tool (a linter, a security scanner, anything that gives you deterministic output) and get it running on a sample file. This is your ground truth before any AI is involved.
3. **Bring in the LLM** : Feed the code (and optionally the static analysis output) to an LLM and have it reason over potential issues, logic bugs, unclear naming, missing edge cases, things a linter wouldn't catch.
4. **Structure your output** : Don't let the model just ramble in prose. Define a clear format for each issue (location, severity, explanation, suggestion) so your output is consistent and easy to act on.
5. **Build an interface** : This can be as simple as a CLI that prints a report, or as ambitious as a GitHub Action that comments directly on a pull request. Pick what's realistic for your week.
6. **Test it on real code** : Run it on something messy, an old project, a deliberately buggy snippet, a real PR, and see if the feedback actually makes sense.
7. **Document what you built** : Capture your design decisions, what worked, what didn't, and what you'd do differently with more time.



Creative Freedom

This challenge is intentionally open on **how** you build it. Some things left entirely up to you:

- Programming language(s) supported
- Which AI models/APIs/tools you use
- Whether it's a CLI, web app, browser extension, or GitHub bot
- Whether it focuses on bugs, security, style, performance, or all of the above
- How "agentic" you want to make it, a single LLM call is a valid starting point; multi-step reasoning or multiple tools is a great stretch goal

Use whatever excites you. The only requirement is that it actually reviews code and gives meaningful feedback.

Timeline

1 week. Scope your project so you have something working end-to-end early, then spend remaining time polishing and adding extras. A simple version that works beats an ambitious version that doesn't.

Deliverables

- A **GitHub repository** containing all your code.
- A [README.md](#) **inside that repository** explaining:
 - What you built and how it works
 - How to run/use it
 - The tools, models, or libraries you relied on
 - At least one example of your agent reviewing real code (before/after, or a sample report)
 - Any challenges you ran into or things you'd improve with more time

Team Format

Work **solo or in a team**, entirely your choice. Team up if you want to tackle a more ambitious version, or go solo if you'd rather move at your own pace.

 **Good luck, and have fun with it.** Share your repo link when you're done, we're excited to see what you build